V3 Grand Madurai

Execution Playbook

Sprint 1 Tasks | Risk Review | Platform Abstraction

Prepared for: V3 Grand Engineering Team
Date: March 2026
Status: Sprint-Ready

Document Overview

This document delivers three complementary analyses for the V3 Grand investment platform, each answering a distinct question:

Part	Question	Deliverable
Part A	How do we execute Sprint 1?	12 developer tasks with files, difficulty, design decisions, and acceptance criteria. Day-by-day plan for 2 engineers. Definition of Done with 5 live demo flows.
Part B	What can go wrong?	Top 10 architecture risks ranked by impact. Each has a concrete mitigation pointing to specific files/modules. Focus: numerical correctness, event ordering, data consistency, security.
Part C	How do we generalize beyond hotels?	AssetPlugin interface separating core platform from asset-specific logic. Hotel plugin implementation. Refactor plan for v3grand-slice. Example: adding Commercial Office with 5 new files and zero core changes.

These three analyses are designed to be used together: Part A gets the team building immediately, Part B de-risks what they are building, and Part C ensures what they build can grow into a multi-asset platform.

Part A — Sprint 1 Execution Guide

Scope: Layer 1 (Single-Deal MVP). Team: 2 senior engineers (Eng-A: full-stack/API, Eng-B: frontend/UI). Duration: 2 weeks (10 working days).

All file paths are relative to v3grand-slice/. Each task includes the design decision and our recommendation — engineers should not need to ask.

Task 1 — Zod Assumption Schemas

Attribute	Detail
Assignee	Eng-A
Difficulty	Small (0.5 day)
Files	packages/core/src/schemas/assumptions.ts (new), packages/core/src/index.ts (add export)
Description	Create Zod schemas for MarketAssumptions and FinancialAssumptions with min/max/step metadata. Each field gets a ZodNumber with .min(), .max(), .describe() carrying the step hint as JSON in the description string.
Design Decision	How to attach slider metadata (min/max/step) to each field?
Recommendation	Use Zod .describe() with a JSON string: z.number().min(0).max(1).describe(JSON.stringify({ step: 0.01, unit: '%', label: 'Base Occupancy' })). The UI parses this at render time. This avoids a separate metadata file and keeps schema and metadata co-located.
Acceptance Criteria	1) Zod.parse succeeds on the existing V3 Grand seed data. 2) Zod.parse rejects occupancy > 1.0, ADR < 0, debtRatio > 1.0, exitMultiple < 1.0. 3) Every field in MarketAssumptions and FinancialAssumptions has a .describe() with valid JSON containing step, unit, and label.

Task 2 — PATCH Assumptions Validation

Attribute	Detail
Assignee	Eng-A
Difficulty	Small (0.5 day)
Files	packages/api/src/routes/deals.ts (modify PATCH handler)
Description	Wire the Zod schemas into the existing PATCH /deals/:id/assumptions route. On validation failure, return 400 with field-level errors. On success, run recompute and return the full DealDashboardView in the response body (not just 200 OK).
Design Decision	Should PATCH accept partial (deep-merge) or full replacement?
Recommendation	Deep merge. Use Zod .deepPartial() on the assumption schemas. Merge the validated patch into the existing deal assumptions with a recursive spread.

	This lets the UI send only changed fields, reducing payload and avoiding accidental overwrites.
Acceptance Criteria	1) PATCH with { marketAssumptions: { baseOccupancy: 0.80 } } merges correctly, other fields unchanged. 2) PATCH with { marketAssumptions: { baseOccupancy: 1.5 } } returns 400 with error: 'baseOccupancy must be <= 1.0'. 3) Response includes full DealDashboardView with recalculated IRR/NPV.

Task 3 — Users Table & Auth Schema

Attribute	Detail
Assignee	Eng-A
Difficulty	Medium (1 day)
Files	packages/db/src/schema/index.ts (add users table + roleEnum), packages/db/src/seed/v3grand.ts (add seed users), packages/db/src/queries/auth.ts (new)
Description	Add users table with id (UUID), email (unique), passwordHash (text), role (enum), createdAt. Seed 5 users (one per role). Add query helpers: getUserByEmail, getUserById, createUser.
Design Decision	Password hashing algorithm?
Recommendation	bcrypt with cost factor 12. Use the bcryptjs npm package (pure JS, no native deps). Store the full bcrypt hash string ($2b$12$...) in passwordHash. Never store plaintext.
Design Decision	Should role be a single value or array?
Recommendation	Single role per user. Multi-role adds complexity with no current need. If a user needs both ANALYST and AUDITOR access, create the ADMIN role as the superuser. We can add role composition in Layer 3.
Acceptance Criteria	1) Seed migration runs idempotently (ON CONFLICT DO NOTHING). 2) getUserByEmail('analyst@v3grand.dev') returns user with role ANALYST. 3) Passwords are bcrypt hashed (verify with bcrypt.compare). 4) All 5 roles in enum: ANALYST, INVESTOR, PM, AUDITOR, ADMIN.

Task 4 — JWT Auth Routes & Guard

Attribute	Detail
Assignee	Eng-A
Difficulty	Medium (1.5 days)
Files	packages/api/src/routes/auth.ts (new), packages/api/src/plugins/auth-guard.ts (new), packages/api/src/index.ts (register plugin + routes)

Description	Implement POST /auth/login (email + password → JWT), GET /auth/me, and a Fastify preHandler plugin that verifies JWT on protected routes. The guard attaches the decoded user to request.user.
Design Decision	JWT claim format?
Recommendation	Payload: { sub: userId, email, role, iat, exp }. Expiry: 24 hours. Algorithm: HS256 with a 256-bit secret from JWT_SECRET env var. No refresh tokens in Sprint 1 — the 24h window is fine for a dev/demo environment. Use @fastify/jwt plugin for clean integration.
Design Decision	Which routes are public vs protected?
Recommendation	Only POST /auth/login is public. Everything else requires a valid JWT. Apply the guard globally in index.ts, then whitelist /auth/login with { preHandler: [] } override.
Acceptance Criteria	1) POST /auth/login with valid credentials returns { token, user: { id, email, role } }. 2) POST /auth/login with wrong password returns 401. 3) GET /deals/:id without token returns 401. 4) GET /deals/:id with valid token returns 200. 5) PATCH /deals/:id/assumptions with INVESTOR role returns 403. 6) PATCH /deals/:id/assumptions with ANALYST role returns 200.

Task 5 — Assumption Editing UI

Attribute	Detail
Assignee	Eng-B
Difficulty	Large (2 days)
Files	packages/ui/src/components/assumptions/AssumptionEditor.tsx (new), packages/ui/src/components/assumptions/AssumptionField.tsx (new), packages/ui/src/hooks/use-assumptions.ts (new), packages/ui/src/app/deals/[dealId]/page.tsx (add tab)
Description	Build a grouped accordion form with 4 sections: Revenue Drivers (occupancy, ADR, growth rates), Operating Costs (6 USALI departments), Debt Structure (ratio, rate, tenor), Exit (multiple, cap rate). Each field renders as a labeled slider + numeric input, using metadata from the Zod schema .describe().
Design Decision	How to handle field debouncing?
Recommendation	Debounce at the field level, not the form level. Use a 600ms debounce on each AssumptionField. On change, fire the mutation with only the changed field path. This gives instant slider feedback while keeping API calls reasonable. Use React Query's useMutation with optimistic updates — update the local cache immediately, rollback on error.
Design Decision	Tab vs sidebar layout for assumptions?
Recommendation	Horizontal tab bar below the deal header: Dashboard | Assumptions. Keep it simple. The assumption panel takes the full content width. No split-screen — switching context is fine for this workflow.

Acceptance Criteria	1) AssumptionEditor renders all fields from the Zod schema with correct min/max/step. 2) Changing a slider updates the numeric input and vice versa. 3) After 600ms idle, a PATCH is fired. 4) Dashboard metrics (IRR, NPV) update after the PATCH returns. 5) Invalid values show inline validation errors (e.g., occupancy > 100% shows red border + message).

Task 6 — Login UI & Auth Context

Attribute	Detail
Assignee	Eng-B
Difficulty	Medium (1 day)
Files	packages/ui/src/context/AuthContext.tsx (new), packages/ui/src/app/login/page.tsx (new), packages/ui/src/app/layout.tsx (wrap with AuthProvider), packages/ui/src/lib/api-client.ts (attach Bearer token)
Description	React context storing JWT + user profile. Login page with email/password form. On login success, store JWT in memory (not localStorage — see decision below), redirect to /deals/:dealId. Unauthenticated users redirect to /login.
Design Decision	Where to store the JWT?
Recommendation	In-memory only (React state via context). Do NOT use localStorage or sessionStorage. Reasons: (1) XSS cannot steal an in-memory token, (2) this is a dev/demo environment so losing the token on refresh is acceptable, (3) we can add httpOnly cookie-based sessions in Layer 3 for production. On page refresh, user logs in again — acceptable for Sprint 1.
Acceptance Criteria	1) /login page renders email + password form. 2) Successful login redirects to /deals/:dealId and shows user's role in the header. 3) Navigating to /deals/:id without login redirects to /login. 4) api-client attaches 'Authorization: Bearer <token>' to all requests. 5) Role is visible in the UI header (e.g., 'Logged in as analyst@v3grand.dev (ANALYST)').

Task 7 — Monthly Revaluation Action

Attribute	Detail
Assignee	Eng-A
Difficulty	Medium (1 day)
Files	packages/api/src/routes/deals.ts (modify POST /underwrite), packages/api/src/services/recompute.ts (add month advancement), packages/core/src/types/engine.ts (add currentMonth to ProFormaInput)
Description	Enhance POST /deals/:id/underwrite to accept { advanceMonth: true }. When true, increment deal.currentMonth by 1 before recompute. The Underwriter uses currentMonth to determine which construction phase the project is in

	(months 0-43 = construction, 44+ = operations). For months during construction, the projection start adjusts accordingly.
Design Decision	How does currentMonth affect the 10-year pro forma?
Recommendation	Keep it simple for Sprint 1: currentMonth is informational metadata stored on the deal and passed into the engine. The Underwriter already starts projections from Year 1 (first full operating year). currentMonth is logged in the audit trail and displayed in the UI. In Layer 2, we will use it to interpolate partial-year adjustments. For now, it advances a counter and triggers a fresh recompute with current assumptions.
Acceptance Criteria	1) POST /underwrite with { advanceMonth: true } increments deal.currentMonth from N to N+1. 2) A new EngineResult and Recommendation are persisted (version N+1). 3) Audit log entry records: 'Monthly revaluation at month N+1. Trigger: manual.' 4) Response includes the new month and full DealDashboardView.

Task 8 — Revaluation History API & UI

Attribute	Detail
Assignee	Eng-B
Difficulty	Medium (1.5 days)
Files	packages/api/src/routes/deals.ts (add GET /deals/:id/history), packages/db/src/queries/index.ts (add getRecommendationHistory), packages/ui/src/components/dashboard/RevalHistory.tsx (new), packages/ui/src/app/deals/[dealId]/page.tsx (add history section)
Description	New API route that returns all recommendations for a deal joined with their engine results, ordered by version DESC. UI component: a table showing version, month, verdict (color-coded badge), confidence bar, IRR, NPV, DSCR, timestamp. Clicking a row could show full details (stretch goal — not required for Sprint 1).
Design Decision	Should history be a separate tab or inline?
Recommendation	Inline on the Dashboard tab, below the metrics strip. A collapsed section titled 'Revaluation History' with an expand toggle. When expanded, shows the last 10 versions in a table. This keeps the most important context (current metrics + trend) on one screen.
Acceptance Criteria	1) GET /deals/:id/history returns array sorted by version DESC. 2) Each entry includes: version, month, verdict, confidence, irrBase, npvBase, dscrAvg, createdAt. 3) UI table renders with verdict badges matching the color scheme in RecommendationCard. 4) After running 3 revaluations, the history table shows all 3 entries.

Task 9 — Revaluation Button & Month Badge

Attribute	Detail

Assignee	Eng-B
Difficulty	Small (0.5 day)
Files	packages/ui/src/app/deals/[dealId]/page.tsx (modify header), packages/ui/src/components/dashboard/RecommendationCard.tsx (add month badge)
Description	Add a 'Run Monthly Revaluation' button in the dashboard header. Shows a confirmation dialog ('Advance to Month N+1 and recompute?'). On confirm, calls POST /underwrite with advanceMonth: true. While running, button shows spinner. After completion, dashboard refreshes. Add a 'Month N' badge next to the verdict in RecommendationCard.
Design Decision	Should we require confirmation before advancing month?
Recommendation	Yes. A simple browser confirm() is fine for Sprint 1. Month advancement is a non-reversible action (we can't decrement currentMonth without data inconsistency). The confirmation dialog should say: 'This will advance the project to Month N+1 and trigger a full revaluation. Continue?'
Acceptance Criteria	1) Button visible to ANALYST and ADMIN roles only (hidden for INVESTOR, PM, AUDITOR). 2) Clicking shows confirmation. 3) On confirm, month advances and dashboard updates. 4) RecommendationCard shows 'Month N' badge. 5) Button is disabled while recompute is in flight.

Task 10 — Role-Based UI Guards

Attribute	Detail
Assignee	Eng-B
Difficulty	Small (0.5 day)
Files	packages/ui/src/components/guards/RoleGate.tsx (new), packages/ui/src/app/deals/[dealId]/page.tsx (wrap mutation controls)
Description	A simple RoleGate component: <RoleGate allow={['ANALYST', 'ADMIN']}>{children}</RoleGate>. Reads current role from AuthContext. If the user's role is not in the allow list, renders nothing (or optionally a tooltip: 'Insufficient permissions'). Wrap: Assumptions tab, Revaluation button, any future mutation controls.
Design Decision	Should unauthorized users see disabled controls or no controls?
Recommendation	Hide entirely. Showing disabled buttons to investors creates confusion ('Why can't I click this?') and implies they should be able to. Clean separation: investors see Dashboard + History (read-only view). Analysts see Dashboard + Assumptions + Revaluation. Admins see everything.
Acceptance Criteria	1) INVESTOR user sees Dashboard tab only (no Assumptions tab, no Revaluation button). 2) ANALYST user sees Dashboard + Assumptions + Revaluation button. 3) ADMIN user sees everything. 4) RoleGate is used consistently — no inline role checks scattered through components.

Task 11 — Integration Smoke Tests

Attribute	Detail
Assignee	Eng-A
Difficulty	Medium (1 day)
Files	packages/api/src/__tests__/smoke.test.ts (new)
Description	A minimal integration test suite that spins up the Fastify server against a test Postgres instance (Docker Compose test profile or Testcontainers). Tests the 3 critical flows end-to-end through the API: login → get dashboard, login → patch assumptions → verify recompute, login → run revaluation → verify history.
Design Decision	Test database strategy?
Recommendation	Use a dedicated test database (v3grand_test) on the same Docker Compose Postgres instance. Before each test suite: drop + recreate schema + run seeds. This is fast enough for 3-5 integration tests and avoids Testcontainers complexity in Sprint 1. Add Testcontainers in Layer 3 for CI.
Acceptance Criteria	1) All 3 flows pass. 2) Tests run in < 10 seconds. 3) Tests are idempotent (can run repeatedly). 4) Tests run with: pnpm test:integration.

Task 12 — Docker Compose Dev Profile & README

Attribute	Detail
Assignee	Eng-A
Difficulty	Small (0.5 day)
Files	packages/infra/docker-compose.yml (add healthchecks), .env.example (new), README.md (update getting started)
Description	Polish the dev environment: add Postgres healthcheck (pg_isready), add .env.example with all required vars documented, update README with: prerequisites, setup steps (pnpm install, docker compose up, pnpm db:seed, pnpm dev), login credentials, and available endpoints.
Acceptance Criteria	1) A new engineer can clone, follow README, and have the full stack running in < 5 minutes. 2) docker compose up -d starts Postgres + Redis with healthchecks. 3) .env.example has every required env var with comments. 4) pnpm dev starts both API and UI in parallel (Turborepo).

Sprint 1 — Day-by-Day Plan

Day	Eng-A (API / Backend)	Eng-B (UI / Frontend)
Day 1	Task 1 (Zod schemas) + Task 2 (PATCH validation)	Task 6 (AuthContext + Login page)

Day 2	Task 3 (Users table + seed)	Task 6 continued + api-client Bearer token
Day 3-4	Task 4 (JWT routes + auth guard)	Task 5 (AssumptionEditor — start)
Day 5	Task 7 (Monthly revaluation API)	Task 5 (AssumptionEditor — complete)
Day 6-7	Task 11 (Integration smoke tests)	Task 8 (Revaluation history UI)
Day 8	Task 12 (Docker polish + README)	Task 9 (Reval button + month badge)
Day 9	Bug fixes + code review	Task 10 (RoleGate) + bug fixes
Day 10	End-to-end demo rehearsal + polish	End-to-end demo rehearsal + polish

Definition of Done — Sprint 1

The sprint is complete when all of the following can be demonstrated live:

- Flow 1 — Login as analyst@v3grand.dev. Navigate to Deal Dashboard. See current verdict (INVEST), IRR (19.8%), NPV (13.38 Cr), DSCR (1.98x). See the 10-year cash flow table.
- Flow 2 — Switch to Assumptions tab. Change Base Occupancy from 0.75 to 0.65 via the slider. Wait 600ms. See dashboard metrics update automatically. IRR drops. Verdict may change to HOLD.
- Flow 3 — Click 'Run Monthly Revaluation'. Confirm dialog. See month advance from 0 to 1. Expand Revaluation History. See two entries: version 1 (original) and version 2 (post-revaluation).
- Flow 4 — Log out. Log in as investor@v3grand.dev. See Dashboard tab only (no Assumptions tab, no Revaluation button). Dashboard metrics are fully visible. History is visible.
- Flow 5 — Log in as admin@v3grand.dev. See all controls. Change an assumption. Run revaluation. Verify full audit trail in history.
- Non-functional — All 3 integration smoke tests pass. New engineer can set up from README in < 5 minutes. No console errors in the browser. No unhandled promise rejections on the server.

Part B — Architecture Risk Review

A critical staff-engineer review of the V3 Grand design across engines, events, workflows, auth, and multi-deal support. Ten risks, ranked by impact, with concrete mitigations pointing to specific files.

Top 10 Risks — Summary

#	Risk	Impact	Likelihood	Category	Mitigation Summary
1	IRR numerical instability	Critical	Medium	Engine	Guard Newton-Raphson with bisection fallback, add golden-file tests against Excel model
2	Event ordering and replay	High	High	Events	Sequence numbers per deal, idempotency keys on all event handlers
3	Recompute race conditions	High	Medium	Workflow	Optimistic locking on deal version, reject stale writes
4	Opex model fragility	High	Medium	Engine	Parameterize department cost model, add invariant assertions
5	JWT secret management	Critical	Low	Security	Rotate via env var, add key-ID claim for multi-key support
6	Investor data isolation	Critical	Medium	Security	Row-level security via deal_access enforced at query layer
7	Event bus in-process SPOF	Medium	High	Events	Write-ahead persistence, dead-letter handling, health endpoint
8	Postgres JSONB query performance	Medium	Medium	Database	Add GIN indexes on hot paths, plan extraction to relational columns
9	UI state desync after failed recompute	Medium	Medium	Frontend	Distinguish recompute failure from network error, show stale-data banner
10	Multi-deal schema migration complexity	Medium	Low	Database	Two-phase migrations, backward-compatible column additions only

Risk 1 — IRR Numerical Instability

Attribute	Detail
Risk	The Newton-Raphson IRR solver can diverge, oscillate, or converge to the wrong root when cash flows have sign changes (common in phased construction projects where FCFE is negative in early years, positive mid-life, then negative again on a terminal refinancing).

Impact	Critical. A wrong IRR silently propagates through Decision gates. An INVEST verdict on a deal that actually has 8% IRR (below WACC) could mean a ₹43 Crore mistake.
Likelihood	Medium. The current V3 Grand base case is well-behaved (one sign change at year 0). But bear scenarios with deep negative FCFE in years 1-3, or deals with a mid-life capex reinvestment, will hit this.
Mitigation	1) Add a bisection-method fallback: if Newton-Raphson fails to converge in 200 iterations or produces a rate outside [-0.5, 5.0], switch to bisection on [-0.99, 10.0] bracket. 2) Add a post-check: verify NPV(IRR) ≈ 0 (within 1e-4). If not, return NaN and flag in engine results. 3) Add golden-file tests: export the V3 Grand Excel model's cash flows and expected IRR. The engine must match within 10 bps.
Where	packages/engines/src/_shared/irr.ts — add bisection fallback. packages/engines/src/underwriter/index.test.ts — add golden-file test with Excel cash flows. packages/engines/src/underwriter/index.ts — add post-check assertion after calcIRR call.

Risk 2 — Event Ordering and Replay

Attribute	Detail
Risk	The Layer 2 event bus persists events then emits them in-process. If the server crashes between persist and emit, events are lost from the in-process queue. If two events fire in rapid succession (e.g., two assumption changes), the second handler may read stale data from the first handler's incomplete write.
Impact	High. Out-of-order or duplicate event handling can produce incorrect engine results or double-counted CAPEX adjustments.
Likelihood	High. This will happen in normal usage — a user rapidly editing two assumption fields will fire two events within milliseconds.
Mitigation	1) Add a monotonic sequence number per deal on the domain_events table. Each event gets dealId + seqNo (auto-increment scoped to deal). Handlers assert they are processing events in order. 2) Add an idempotency key to each event (hash of dealId + eventType + payload). Handlers check: if this idempotency key exists in a processed_events set (Redis or a DB column), skip. 3) Add a startup recovery job: on server boot, scan domain_events for unprocessed events (status = PENDING) and replay them in seqNo order. 4) Serialize event processing per deal: use a per-deal mutex (in-memory Map<dealId, Promise>) so only one handler runs at a time for a given deal.
Where	packages/db/src/schema/index.ts — add seqNo, idempotencyKey, status columns to domain_events. packages/api/src/event-bus.ts — add per-deal serialization mutex and idempotency check. packages/api/src/index.ts — add startup recovery scan.

Risk 3 — Recompute Race Conditions

Attribute	Detail
Risk	Two concurrent PATCH /assumptions requests can both read the same deal version, merge their changes, and write back — causing one set of changes to be lost (last-write-wins). Worse, both trigger recompute, and the second recompute may overwrite the first's engine results.
Impact	High. Silent data loss on assumptions. Engine results may not reflect the latest state of the deal.
Likelihood	Medium. Less likely with a single user, but probable when PM and Analyst are both editing simultaneously, or when a rapid UI debounce fires twice.
Mitigation	1) Optimistic concurrency control: add a version column (integer) to the deals table. Every PATCH must include the expected version in the request body. The UPDATE query uses WHERE version = expected. If 0 rows updated, return 409 Conflict with the current version. 2) The UI retries on 409: re-fetch the deal, re-apply the user's change to the latest state, re-submit. React Query's onError can handle this automatically. 3) Recompute service checks the deal version at start and end. If it changed during compute, discard results and re-run.
Where	packages/db/src/schema/index.ts — add version column to deals (default 1). packages/db/src/queries/index.ts — updateDealAssumptions uses WHERE version = $expected, returns updated row count. packages/api/src/routes/deals.ts — PATCH handler checks row count, returns 409 if 0. packages/ui/src/hooks/use-assumptions.ts — add 409 retry logic.

Risk 4 — Opex Model Fragility

Attribute	Detail
Risk	The opex model uses costPctOfRevenue per USALI department. We already fixed the 0.97 sum bug (now 0.49), but the model is fragile: adding a department, changing the cost basis (% of revenue vs. fixed cost vs. per-key), or modeling seasonality requires engine code changes.
Impact	High. Any new deal with a different cost structure (e.g., a resort with higher F&B, or a budget hotel with no banquets) requires modifying the Underwriter, which should be a pure configuration change.
Likelihood	Medium. Will definitely happen when generalizing to non-V3-Grand deals.
Mitigation	1) Add an invariant assertion in the Underwriter: sum of all department costPctOfRevenue must be < 0.85. If it exceeds this, the engine throws with a clear message. 2) Add a department cost model type: 'pct_of_revenue' | 'fixed_annual' | 'per_key_per_month'. The Underwriter branches on this type. For Sprint 1, only pct_of_revenue is implemented; the other two throw NotImplementedError. 3) Add a Underwriter config validation step at the top of the engine: check that all required departments exist, no negative costs, total cost ratio is reasonable.
Where	packages/core/src/types/deal.ts — add costBasis field to USALIDepartment type. packages/engines/src/underwriter/index.ts — add invariant assertion on cost sum, branch on costBasis type.

	packages/engines/src/underwriter/index.test.ts — add test: opex sum > 0.85 throws.

Risk 5 — JWT Secret Management

Attribute	Detail
Risk	JWT_SECRET is a single env var. If compromised, all tokens are valid until the secret is rotated. Rotation invalidates ALL existing sessions (hard logout for every user).
Impact	Critical. A leaked secret means full account takeover for any user.
Likelihood	Low in Sprint 1 (dev environment), but will be Medium in production.
Mitigation	1) For Sprint 1: enforce minimum secret length (32 chars) at startup — fail fast if shorter. 2) For Layer 3: add a kid (key ID) claim to each JWT. Support two active secrets (current + previous). On verification, try current first, fall back to previous. This allows graceful rotation: deploy new secret → existing tokens still work on previous → after 24h (token expiry), remove previous. 3) Document the rotation procedure in the README.
Where	packages/api/src/config.ts — validate JWT_SECRET length >= 32. packages/api/src/plugins/auth-guard.ts — Layer 3: add kid-based multi-key verification.

Risk 6 — Investor Data Isolation

Attribute	Detail
Risk	In the multi-deal Layer 3, investors should only see deals they have access to. Currently, deal access is checked at the route level (API middleware), but a bug in any route handler could leak data from other deals. There is no defense-in-depth at the database layer.
Impact	Critical. An investor seeing another partnership's financial details is a breach of confidentiality and potentially a legal issue.
Likelihood	Medium. As the API grows (Layer 2 adds 8+ new routes), the surface area for authorization bugs increases.
Mitigation	1) Enforce deal access at the query layer, not just routes. Every query function in packages/db/src/queries/ takes userId as a required parameter and joins against deal_access. No query can return data for a deal the user cannot access. 2) Add a getDealIdsForUser(userId) helper. All queries use: WHERE dealId IN (SELECT dealId FROM deal_access WHERE userId = $userId). 3) For Sprint 1 (single deal): add the deal_access table and junction NOW, even before multi-deal. This establishes the pattern early. Seed all 5 users with access to the V3 Grand deal. 4) Add an integration test: create two deals, grant user access to only one, verify queries for the second deal return empty results.
Where	packages/db/src/schema/index.ts — add deal_access table in Sprint 1 (Task 3). packages/db/src/queries/index.ts — add userId parameter to all deal-

	scoped queries. packages/api/src/__tests__/smoke.test.ts — add isolation test.

Risk 7 — Event Bus In-Process SPOF

Attribute	Detail
Risk	The Layer 2 event bus is in-process (Node.js EventEmitter). If the Fastify server crashes, all pending event handlers are lost. If the handler itself throws, the event is silently dropped.
Impact	Medium. A dropped change_order.approved event means CAPEX is updated but engines are not recomputed. The dashboard shows stale metrics.
Likelihood	High. In-process event handling WILL fail eventually — OOM, uncaught exception, or deployment restart.
Mitigation	1) Write-ahead: persist event to domain_events with status PENDING before emitting to in-process handlers. Handler updates status to PROCESSED on success, FAILED on error. 2) Dead-letter handling: events with status FAILED or PENDING older than 5 minutes are retried by a polling interval (setInterval, 60s). After 3 retries, status becomes DEAD_LETTER and an alert is created. 3) Health endpoint: GET /health includes { eventBus: { pendingCount, deadLetterCount, lastProcessedAt } }. If pendingCount > 10 or deadLetterCount > 0, the health check returns 503 for alerting. 4) Plan for Layer 3+: if event volume or reliability requirements increase, swap the in-process emitter for Redis Streams. The EventBus interface stays the same — only the transport changes.
Where	packages/db/src/schema/index.ts — add status, retryCount columns to domain_events. packages/api/src/event-bus.ts — implement write-ahead + retry loop. packages/api/src/routes/health.ts (new) — health endpoint with event bus stats.

Risk 8 — Postgres JSONB Query Performance

Attribute	Detail
Risk	The deals table stores property, marketAssumptions, financialAssumptions, capexPlan, and opexModel as JSONB columns. This is convenient for rapid iteration but creates performance risk: any query that filters or sorts by a nested JSONB field (e.g., 'find all deals with IRR > 15%') requires a full table scan or GIN index.
Impact	Medium. Not a problem with 1-10 deals. Becomes painful at 100+ deals with complex filters on the portfolio dashboard.
Likelihood	Medium. Will surface in Layer 3 when the portfolio dashboard needs to list and filter deals by latest metrics.
Mitigation	1) For Sprint 1: no change needed — single deal, no filtering. 2) For Layer 3: add GIN indexes on the JSONB columns that are queried: CREATE INDEX idx_deals_market ON deals USING GIN (market_assumptions). 3) For the portfolio dashboard: do NOT query into JSONB. Instead, maintain a

	deals_summary materialized view (or denormalized columns) with: dealId, name, status, latestVerdict, latestIRR, latestNPV, latestDSCR, updatedAt. The recompute service updates this after each run. The portfolio GET /deals query hits this flat table. 4) Add a migration plan: when we need to extract a JSONB field to a relational column, use a two-phase approach: add column → backfill → switch queries → drop JSONB field.
Where	packages/db/src/schema/index.ts — Layer 3: add deals_summary table or materialized view. packages/api/src/services/recompute.ts — Layer 3: update deals_summary after recompute. packages/api/src/routes/deals.ts — Layer 3: GET /deals queries deals_summary, not deals.

Risk 9 — UI State Desync After Failed Recompute

Attribute	Detail
Risk	If recompute fails (engine error, DB timeout, or network issue), the UI may show stale metrics while the deal's assumptions have already been updated. The user thinks the dashboard reflects their latest changes, but it does not.
Impact	Medium. Could lead to incorrect investment decisions based on stale data.
Likelihood	Medium. Engine errors are rare in the happy path but will occur with extreme assumption values (e.g., 0% occupancy, 100x exit multiple).
Mitigation	1) The PATCH /assumptions response should include a status field: 'recomputed' | 'recompute_failed'. If failed, include the error message (sanitized). 2) The UI shows a 'Stale Data' amber banner when the last recompute failed. Banner includes: 'Metrics may not reflect your latest changes. Engine error: [message]. Click to retry.' 3) The retry button calls POST /underwrite (manual recompute). 4) The RecommendationCard shows a warning icon if the engine_result status is FAILED.
Where	packages/api/src/services/recompute.ts — wrap engine calls in try/catch, return status. packages/api/src/routes/deals.ts — include recompute status in PATCH response. packages/ui/src/components/dashboard/StaleDataBanner.tsx (new) — amber banner component. packages/ui/src/hooks/use-assumptions.ts — check response status, set stale flag.

Risk 10 — Multi-Deal Schema Migration Complexity

Attribute	Detail
Risk	Transitioning from single-deal (hardcoded V3_GRAND_DEAL_ID) to multi-deal requires schema changes that could break existing data: adding deal_access junction table, adding deals_summary, potentially splitting JSONB into relational columns.
Impact	Medium. A botched migration could corrupt the seed deal or lose engine result history.
Likelihood	Low. Drizzle Kit handles migrations well, and the dataset is small.

Mitigation	1) Establish migration discipline NOW (Sprint 1): all schema changes go through drizzle-kit generate. Never hand-edit the schema without a migration file. 2) Backward-compatible only: Layer 3 migrations must only ADD columns/tables, never remove or rename. Removal happens in a subsequent release after code no longer references the old column. 3) Add a pre-migration backup step to the README: pg_dump before running migrations. 4) All seed data uses ON CONFLICT DO NOTHING — re-running seeds is safe. 5) Add a migration test: run all migrations on an empty database, then verify seed data loads correctly.
Where	packages/db/drizzle.config.ts — configure migrations output directory. packages/db/package.json — add scripts: db:generate, db:migrate, db:seed, db:backup. README.md — document migration procedure.

Part C — Asset-Class Abstraction Layer

Design the separation between the core Deal Lifecycle Platform and asset-specific plug-ins, starting with hotel and enabling future asset classes (commercial office, co-living, warehouse, mixed-use).

C.1 Hotel-Specific Code Inventory

Every hotel-specific type, field, and calculation in the current v3grand-slice, with its location and proposed treatment.

Item	Current Location	Specific?	Proposed Treatment
Property.keyCount, phaseKeys	core/types/deal.ts	Hotel	Move to asset-specific config. Generic: Property has location, area, assetConfig: JSONB.
MarketAssumptions (occupancy, ADR, RevPAR)	core/types/deal.ts	Hotel	Replace with AssetPlugin.marketAssumptions type. Hotel plugin defines occupancy/ADR. Office plugin would define rent/sqft, vacancy.
OpexModel with USALI departments	core/types/deal.ts	Hotel	Replace with AssetPlugin.operatingModel type. Hotels use USALI departments. Office uses OPEX categories (CAM, insurance, management).
managementFeePct, incentiveFeePct, ffAndEReservePct	core/types/deal.ts	Hotel	Move to hotel plugin. These are hotel management contract terms. Office/warehouse have different fee structures.
Occupancy ramp array	core/types/deal.ts	Hotel	Move to hotel plugin as stabilizationCurve. Office plugin would have a different lease-up curve.
FinancialAssumptions (debtRatio, rate, tenor, exit multiple)	core/types/deal.ts	Generic	Keep in core. Debt structure and exit valuation apply to all real estate asset classes.
CapexPlan	core/types/deal.ts	Generic	Keep in core. All asset classes have CAPEX.
Partnership, Partner	core/types/deal.ts	Generic	Keep in core. JV structures are asset-agnostic.
Underwriter engine (10-yr hotel cash flow)	engines/underwriter	Hotel	Refactor to implement AssetPlugin.buildProForma(). Revenue model (keys × occ × ADR) is hotel-specific. Debt

			service and exit value calc stay in core.
USALI cost calculation	engines/underwriter	Hotel	Part of hotel buildProForma(). Office would have a different cost model (NNN lease vs. gross lease).
Decision engine (6 gates)	engines/decision	Generic	Keep in core. Gate thresholds are configurable per deal, not per asset class. All asset classes need IRR/NPV/DSCR evaluation.
ProFormaOutput (yearProjections)	core/types/engine.ts	Generic	Keep in core. The shape of year-by-year projections (revenue, costs, NOI, EBITDA, FCFE) is universal across asset classes.
UI dashboard components	ui/components	Mixed	MetricsStrip and CashFlowTable are generic. RecommendationCard is generic. Add an AssetPlugin.renderKeyMetrics() hook for asset-specific KPIs.

C.2 Core Platform vs Asset Plugin

The clean separation into two layers:

Core Deal Lifecycle Platform (asset-agnostic)

- Deal entity: id, name, status, lifecyclePhase, currentMonth, version, partnership, financialAssumptions, capexPlan.
- Engine framework: ProFormaOutput shape (yearly projections with standardized fields), Decision engine (configurable gates), engine result versioning.
- Workflow infrastructure: event bus, recompute service, audit logging.
- Auth and access control: users, roles, deal_access.
- UI shell: deal dashboard layout, metrics strip, cash flow table, recommendation card, revaluation history.
- Construction tracking: budget lines, milestones, RFIs, change orders (asset-agnostic — all construction projects have these).

Asset Plugin (hotel, office, co-living, etc.)

- Asset-specific types: market assumptions shape, operating model shape, asset configuration.
- Revenue model: the function that calculates gross revenue from assumptions (hotel: keys × occupancy × ADR; office: sqft × rent/sqft × (1 - vacancy)).
- Cost model: the function that calculates operating expenses from the operating model.

- Stabilization curve: how the asset ramps to steady-state (hotel: occupancy ramp; office: lease-up curve).
- UI KPI overrides: asset-specific headline metrics (hotel: RevPAR, GOP%; office: NOI/sqft, cap rate).
- Seed data templates: default assumptions for a new deal of this asset class.

C.3 AssetPlugin Interface

The TypeScript interface that every asset plugin must implement. This is the contract between the core platform and asset-specific logic.

```
// packages/core/src/types/asset-plugin.ts

import { ProFormaOutput, YearProjection } from './engine';
import { FinancialAssumptions, CapexPlan, Partnership } from './deal';

/**
 * Every asset class implements this interface.
 * The core platform calls these hooks during deal creation,
 * underwriting, and dashboard rendering.
 */
export interface AssetPlugin<
  TMarket = unknown,     // Asset-specific market assumptions
  TOpex = unknown,       // Asset-specific operating model
  TConfig = unknown,     // Asset-specific configuration (keys, sqft, etc.)
> {
  /** Unique identifier for this asset class */
  readonly assetClass: string; // 'hotel' | 'office' | 'co-living' | ...

  /** Human-readable label */
  readonly label: string; // 'Hotel / Resort' | 'Commercial Office' | ...

  // ── Type Guards ──

  /** Zod schema for validating market assumptions */
  marketSchema: ZodSchema<TMarket>;

  /** Zod schema for validating operating model */
  opexSchema: ZodSchema<TOpex>;

  /** Zod schema for validating asset configuration */
  configSchema: ZodSchema<TConfig>;

  // ── Pro Forma Hooks ──

  /**
   * Build one year of the pro forma projection.
   * Called by the core Underwriter for each year in the hold period.
   * Must return revenue, costs, and asset-specific metrics.
   */
  buildYearProjection(input: {
    year: number;              // 1-indexed operating year
    config: TConfig;           // e.g., { totalKeys: 72, phase1: 55 }
    market: TMarket;           // e.g., { baseOccupancy: 0.75, adr: 4200 }
    opex: TOpex;               // e.g., { departments: [...] }
    financials: FinancialAssumptions;
    capex: CapexPlan;
    prevYear?: YearProjection; // Previous year for growth calcs
  }): YearProjection;
```

```
  /**
   * Compute the exit (terminal) value at the end of the hold period.
   * Default: exitMultiple × final year EBITDA. Override for cap-rate.
   */
  computeExitValue(input: {
    finalYear: YearProjection;
    financials: FinancialAssumptions;
  }): number;

  /**
   * Return asset-specific KPI labels and values for the dashboard.
   * Example hotel: [{ label: 'RevPAR', value: 3150, format: 'currency' }]
   */
  getKeyMetrics(projection: YearProjection): AssetKPI[];

  /**
   * Default assumptions for a new deal of this asset class.
   * Used by the deal creation wizard.
   */
  getDefaultAssumptions(): {
    market: TMarket;
    opex: TOpex;
    config: TConfig;
  };
}

export interface AssetKPI {
  label: string;
  value: number;
  format: 'currency' | 'percent' | 'number' | 'multiple';
  unit?: string; // '₹' | '%' | 'x'
}
```

C.4 Hotel Plugin — Implementation

How the existing Underwriter maps onto the AssetPlugin interface with minimal rewrites.

```
// packages/engines/src/plugins/hotel.ts

import { AssetPlugin, AssetKPI } from '@v3grand/core';
import { HotelMarket, HotelOpex, HotelConfig } from './hotel-types';
import { hotelMarketSchema, hotelOpexSchema, hotelConfigSchema }
  from './hotel-schemas';

export const hotelPlugin: AssetPlugin<HotelMarket, HotelOpex, HotelConfig> = {
  assetClass: 'hotel',
  label: 'Hotel / Resort',

  marketSchema: hotelMarketSchema,
  opexSchema: hotelOpexSchema,
  configSchema: hotelConfigSchema,

  buildYearProjection({ year, config, market, opex, financials, prevYear }) {
    // This is the EXISTING logic from underwriter/index.ts, extracted:
    const totalKeys = year >= config.phase2StartYear
      ? config.phase1Keys + config.phase2Keys
      : config.phase1Keys;

    const occupancy = market.occupancyRamp[year] ?? market.baseOccupancy;
    const adr = year <= market.stabilizationYear
      ? market.baseADR + (market.stabilizedADR - market.baseADR)
        * (year / market.stabilizationYear)
      : (prevYear?.adr ?? market.stabilizedADR)
```

```
        * (1 + market.adrGrowthRate);

    const roomNights = totalKeys * 365 * occupancy;
    const revenue = roomNights * adr;

    // USALI department costs (existing logic)
    const totalOpex = opex.departments.reduce(
      (sum, d) => sum + revenue * d.costPctOfRevenue, 0);
    const gop = revenue - totalOpex;

    // Management + incentive + FF&E fees
    const mgmtFee = revenue * market.managementFeePct;
    const incentiveFee = gop > 0 ? gop * market.incentiveFeePct : 0;
    const ffeFee = revenue * market.ffAndEReservePct;
    const ebitda = gop - mgmtFee - incentiveFee - ffeFee;

    return {
      year, totalKeys, occupancy, adr,
      revPAR: revenue / (totalKeys * 365),
      revenue, totalOpex, gop,
      gopMargin: revenue > 0 ? gop / revenue : 0,
      ebitda, ebitdaMargin: revenue > 0 ? ebitda / revenue : 0,
      debtService: 0, // filled by core Underwriter
      fcfe: 0,        // filled by core Underwriter
    };
  },

  computeExitValue({ finalYear, financials }) {
    return finalYear.ebitda * financials.exitMultiple;
  },

  getKeyMetrics(proj) {
    return [
      { label: 'RevPAR', value: proj.revPAR, format: 'currency', unit: '₹' },
      { label: 'Occupancy', value: proj.occupancy, format: 'percent' },
      { label: 'ADR', value: proj.adr, format: 'currency', unit: '₹' },
      { label: 'GOP %', value: proj.gopMargin, format: 'percent' },
    ];
  },

  getDefaultAssumptions: () => ({ /* V3 Grand seed values */ }),
};
```

C.5 Core Underwriter Refactor

The Underwriter becomes a thin orchestrator that delegates asset-specific logic to the plugin.

```
// packages/engines/src/underwriter/index.ts (refactored)

import { AssetPlugin } from '@v3grand/core';
import { calcIRR, calcNPV } from '../_shared/irr';

export function runUnderwriter(
  input: ProFormaInput,
  plugin: AssetPlugin  // <-- NEW: injected by caller
): ProFormaOutput {
  const { financials, capex, holdPeriod = 10 } = input;
  const totalInvestment = capex.totalInvestment;
  const equityInvested = totalInvestment * (1 - financials.debtRatio);
  const debtAmount = totalInvestment * financials.debtRatio;

  // Core: compute annual debt service (asset-agnostic)
  const annualDebtService = computeAnnuity(
```

```
    debtAmount, financials.interestRate, financials.debtTenorYears);

  // Delegate year-by-year to the plugin
  const projections: YearProjection[] = [];
  for (let yr = 1; yr <= holdPeriod; yr++) {
    const proj = plugin.buildYearProjection({
      year: yr,
      config: input.assetConfig,   // JSONB, plugin casts
      market: input.marketAssumptions,
      opex: input.operatingModel,
      financials,
      capex,
      prevYear: projections[yr - 2],
    });
    // Core fills in debt service and FCFE
    proj.debtService = yr <= financials.debtTenorYears
      ? annualDebtService : 0;
    proj.fcfe = proj.ebitda - proj.debtService;
    projections.push(proj);
  }

  // Core: exit value via plugin hook
  const exitValue = plugin.computeExitValue({
    finalYear: projections[holdPeriod - 1],
    financials,
  });

  // Core: IRR, NPV, equity multiple (asset-agnostic)
  const cashFlows = [-equityInvested,
    ...projections.map((p, i) =>
      i === holdPeriod - 1 ? p.fcfe + exitValue : p.fcfe)];
  const irr = calcIRR(cashFlows);
  const npv = calcNPV(cashFlows, financials.wacc);

  return { projections, irr, npv, exitValue, /* ... */ };
}
```

Key point: the refactored Underwriter is ~50 lines of asset-agnostic orchestration. All hotel math moved to hotelPlugin.buildYearProjection(). Adding a new asset class means writing a new plugin file — zero changes to the core Underwriter.

C.6 Decision Engine — Already Generic

The Decision engine evaluates gates on ProFormaOutput (IRR, NPV, DSCR, equity multiple, payback). These metrics are asset-agnostic — they come from the standardized ProFormaOutput shape that all plugins produce. No changes needed.

The only extension point: asset plugins can contribute additional risk flags via a getAdditionalRiskFlags(projection) method. For hotels, this could flag: occupancy below 60%, ADR below market average. For office, it could flag: vacancy above 20%, tenant concentration > 30%.

C.7 UI Plugin Hooks

The dashboard uses plugin.getKeyMetrics() to render asset-specific KPIs alongside the universal metrics (IRR, NPV, DSCR).

UI Component	Generic or Plugin?	Notes

MetricsStrip (IRR, NPV, DSCR, Eq. Mult)	Generic	Always shown. Comes from ProFormaOutput.
Asset KPI row (RevPAR, Occupancy, ADR, GOP%)	Plugin	Rendered by mapping plugin.getKeyMetrics() into MetricCard components.
CashFlowTable	Generic	Columns: year, revenue, costs, GOP, EBITDA, debt service, FCFE. Asset plugins can inject additional columns via an optional getExtraColumns() hook.
RecommendationCard	Generic	Verdict, confidence, gates — all asset-agnostic.
AssumptionEditor	Plugin	Field definitions come from plugin.marketSchema and plugin.opexSchema. The editor renders dynamically based on Zod schema metadata.

C.8 Required Refactors in v3grand-slice

Changes needed to avoid hard-coding 'hotel' into the platform. These should be done in Layer 3 before adding a second asset class.

File	Refactor
packages/core/src/types/deal.ts	Replace MarketAssumptions, OpexModel, and Property.keyCount with generic JSONB fields: deal.marketAssumptions (any — validated by plugin), deal.operatingModel (any — validated by plugin), deal.assetConfig (any — validated by plugin). Keep FinancialAssumptions, CapexPlan, Partnership as-is (generic). Add assetClass: string field to Deal.
packages/core/src/types/engine.ts	Add optional assetKPIs: AssetKPI[] to YearProjection. The plugin fills these; the core Underwriter passes them through.
packages/engines/src/underwriter/index.ts	Refactor to accept AssetPlugin parameter (see C.5). Move hotel-specific revenue/cost logic to hotel plugin. Keep debt service, exit value, IRR/NPV/DSCR in core.

packages/engines/src/index.ts	Add a plugin registry: Map<string, AssetPlugin>. Export getPlugin(assetClass): AssetPlugin. Register hotelPlugin at import time.
packages/api/src/services/recompute.ts	Load the appropriate plugin via getPlugin(deal.assetClass) before calling Underwriter. Pass plugin to runUnderwriter().
packages/api/src/routes/deals.ts	PATCH /assumptions validation: load plugin, use plugin.marketSchema and plugin.opexSchema for validation instead of hard-coded Zod schemas.
packages/db/src/schema/index.ts	No structural change — JSONB columns already support arbitrary shapes. Add an assetClass column (text, not null, default 'hotel') to the deals table for querying.
packages/db/src/seed/v3grand.ts	Add assetClass: 'hotel' to the seed deal. Otherwise unchanged.
packages/ui/src/components/dashboard/MetricsStrip.tsx	Split into two rows: (1) universal metrics from ProFormaOutput, (2) asset KPIs from plugin.getKeyMetrics(). Render row 2 dynamically.
packages/ui/src/components/assumptions/AssumptionEditor.tsx	Already designed to render from Zod schema metadata. No change needed — it will automatically render hotel or office fields based on the plugin's schema.

C.9 Adding a Second Asset Class — Example: Commercial Office

Once the refactors in C.8 are done, adding a new asset class requires exactly these files:

New File	Content
packages/engines/src/plugins/office-types.ts	OfficeConfig { totalSqFt, floors, parkingSpots }. OfficeMarket { baseRentPerSqFt, vacancy, escalationRate, camRecoveryPct }. OfficeOpex { propertyTax, insurance, management, utilities, janitorial }.
packages/engines/src/plugins/office-schemas.ts	Zod schemas with min/max/step metadata for the assumption editor.
packages/engines/src/plugins/office.ts	Implements AssetPlugin<OfficeMarket, OfficeOpex, OfficeConfig>. buildYearProjection: revenue = leasableSqFt ×

	rent × (1 - vacancy). Costs = NNN recovery model. getKeyMetrics: NOI/sqft, cap rate, vacancy %.
packages/engines/src/index.ts	Register officePlugin in the plugin map.
packages/db/src/seed/office-template.ts	Default assumptions for a new office deal.

Zero changes to: Decision engine, event bus, recompute service, auth, construction tracking, UI shell, dashboard layout, cash flow table, recommendation card. This is the power of the plugin architecture.